Broward International University

HUMAN COMPUTER INTERACTION, AND DIGITAL CITIZENSHIP (CSE6061)

Dr. Walter Marcelo Fuertes Díaz

Tarea de Asignación 2 - Prototipo de alta fidelidad del Knowledge Base Portal: diseño, desarrollo e implementación

Presentado por:

Diana Bello

Miami, octubre de 2026

Índice

1 Introducción 2

2 Fase de diseño 2

2.1 Diseño conceptual 2

2.2 Arquitectura de software 4

2.3 Modelo relacional de la base de datos 5

2.4 Evolución del mockup al prototipo 5

3 Fase de desarrollo 7

3.1 Backend 7

3.2 Frontend 8

4 Implementación y evaluación 10

4.1 Implementación 10

4.2 Evaluación del prototipo 10

5 Conclusiones 11

6 Referencias 12

1 Introducción

El presente informe documentó las fases de diseño, desarrollo e implementación del prototipo de alta fidelidad del Knowledge Base Portal, correspondiente a la Tarea de Asignación 2 de la asignatura CSE661.

En este contexto, el portal se concibió como un sistema web interno que centraliza la documentación técnica y funcional de soluciones de software: complementos de ERP, plugins de punto de venta e integraciones. De acuerdo con Rogers et al. (2023), un prototipo de alta fidelidad se asemeja al producto final en apariencia e interacción, y permite evaluar la experiencia de usuario con datos reales. Por consiguiente, el trabajo partió del wireframe elaborado en Figma y del mockup elaborado en Axure RP, ambos entregados en asignaciones previas, y los convirtió en una aplicación con operaciones CRUD sobre una base de datos relacional.

El objetivo del prototipo consistió en validar la experiencia de usuario mediante pruebas controladas sobre una solución en funcionamiento. Asimismo, el código fuente, los recursos visuales, la documentación técnica y los resultados de las pruebas se organizaron en un repositorio de GitHub, con cuatro carpetas que corresponden a las fases de la tarea: Design, Backend, Frontend y Testing.

Por otra parte, el alcance excluyó el inicio de sesión único con Microsoft previsto para el producto final. En su lugar, se implementó una autenticación local con registro abierto, acorde con el contexto de evaluación académica. De esta manera, las secciones siguientes describen el diseño, el desarrollo del backend y del frontend, y la evaluación del prototipo.

2 Fase de diseño

2.1 Diseño conceptual

El diseño conceptual definió tres perfiles de usuario con permisos acumulativos, tal como se muestra en la Tabla 1. En este contexto, cada perfil amplía los permisos del anterior, de modo que la interfaz presenta a cada usuario solo las acciones que le corresponden. Además, toda cuenta

nueva se registró con el perfil de consultor, y el perfil de administrador se creó mediante un script de carga inicial con credenciales tomadas de las variables de entorno.

Tabla 1

Perfiles de usuario y permisos

Rol	**Etiqueta en la interfaz**	**Permisos**
consultant	Consultor	Consultar el catálogo, las fichas y la documentación; descargar instalables.
author	Autor técnico	Permisos del consultor, más crear, editar y eliminar soluciones, responsables, componentes y secciones.
admin	Administrador	Permisos del autor técnico, más listar usuarios, cambiar su rol y eliminarlos.

Asimismo, la solución se definió como el objeto central del modelo conceptual. Cada solución reúne sus responsables (un Product Owner y uno o más desarrolladores), sus componentes con versión e instalable, y hasta cuatro secciones de documentación: técnica, novedades, guía de usuario y ficha comercial. Por consiguiente, la navegación del portal siguió la jerarquía catálogo, ficha de la solución y sección de documentación, ya presente en el wireframe y en el mockup. Además, el portal incorporó un indicador de cobertura con el porcentaje de soluciones que tienen la documentación completa.

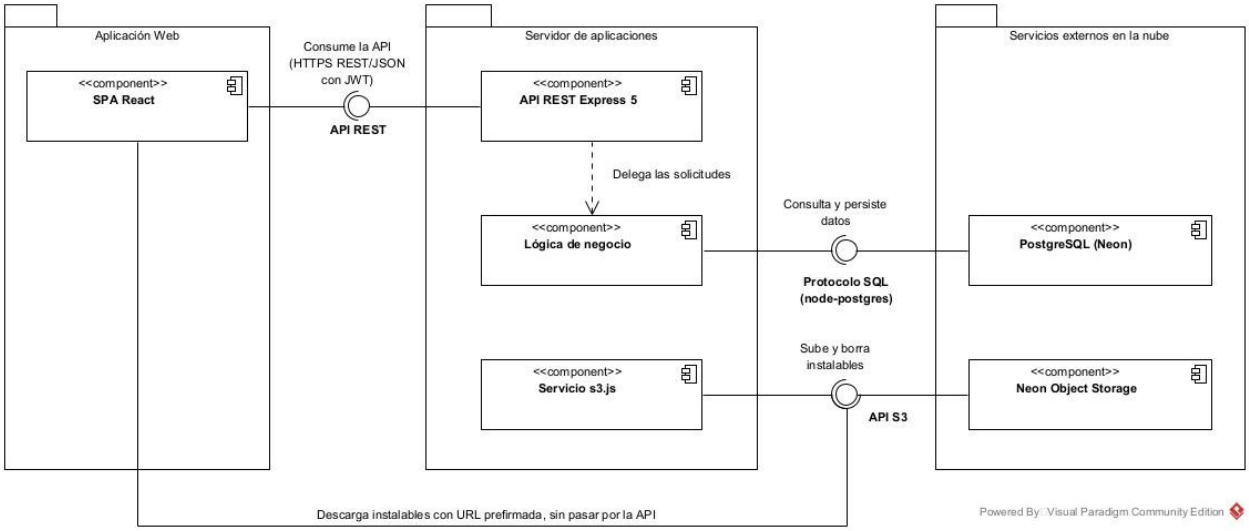
2.2 Arquitectura de software

La arquitectura adoptó un modelo cliente-servidor de tres capas, representado en el diagrama de componentes de la Figura 1. En este contexto, la aplicación web es una SPA en React que consume una API REST en Express 5 mediante HTTPS y JSON, con un JSON Web Token en cada petición.

De acuerdo con el estilo arquitectónico REST, cada recurso se expuso con una URL propia y con los métodos HTTP estándar. A su vez, la API delega las solicitudes en la lógica de negocio, que persiste los datos en PostgreSQL alojado en Neon y gestiona los instalables en Neon Object Storage mediante la API de S3.

Figura 1

Diagrama de componentes del Knowledge Base Portal



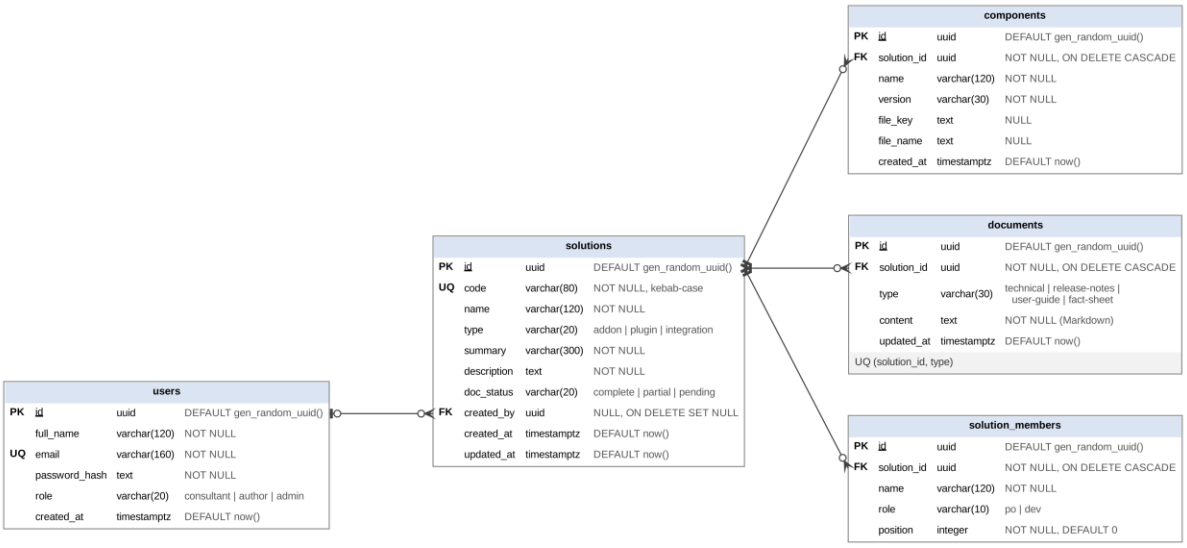
Una decisión de la arquitectura consistió en descargar los instalables con una URL prefirmada, sin que el archivo pase por la API. De esta manera, el servidor no transfiere archivos de hasta 50 MB y el bucket permanece privado. Asimismo, el servicio de almacenamiento se escribió con el SDK estándar de AWS. Por consiguiente, la migración a Amazon S3 requiere solo el cambio de las variables de entorno, sin modificar el código.

2.3 Modelo relacional de la base de datos

El modelo relacional de la Figura 2 contiene cinco tablas con claves primarias UUID. En este contexto, la tabla solutions ocupa la posición central y se relaciona de uno a muchos con components, documents y solution_members, con borrado en cascada. Por su parte, users se relaciona con solutions mediante la columna created_by, que toma el valor nulo si se elimina el usuario. Asimismo, las restricciones del esquema aplican reglas de negocio: un código único en formato kebab-case por solución, una sola fila por tipo de sección y un único Product Owner por solución. Por consiguiente, la base de datos impide estados inconsistentes aun si una validación previa falla.

Figura 2

Modelo relacional de la base de datos en PostgreSQL



2.4 Evolución del mockup al prototipo

El prototipo conservó la estructura, la jerarquía y la ubicación de los elementos definidas en el mockup; sin embargo, se introdujeron cambios justificados por principios de usabilidad y accesibilidad. La Tabla 2 resume esos cambios y el principio que motivó cada uno. Asimismo, las vistas sin referencia en el mockup, como el registro, los formularios de gestión y la administración

de usuarios, se diseñaron con el mismo lenguaje visual de tarjetas, tipografía y botones. De esta manera, se cumplió la heurística de consistencia y estándares (Nielsen, 2024).

Tabla 2

Cambios respecto del mockup y principio que los justifica

Mockup	**Prototipo**	**Principio**
Botón «Continuar con Microsoft».	Formulario de correo y contraseña, con enlace al registro.	Alcance del contexto de evaluación académica.
Texto secundario en #666666.	Texto secundario en #595959.	Contraste mínimo AA (Kirkpatrick et al., 2018).
Sin estados de carga ni de resultado.	Skeletons, botones con «Guardando…» y avisos tras cada acción.	Visibilidad del estado del sistema (Nielsen, 2024).
Botones y enlaces sin tamaño mínimo.	Objetivos de 44 px o más.	Ley de Fitts (Fitts, 1954).
Eliminación sin confirmación.	Modal de confirmación y aviso de cambios sin guardar.	Prevención de errores (Nielsen, 2024).
Tabla de contenidos fija a la derecha.	Tabla plegable en pantallas desde 360 px.	Diseño responsivo (Kirkpatrick et al., 2018).

3 Fase de desarrollo

3.1 Backend

El backend se desarrolló en Node.js con Express 5 y se organizó en capas de rutas, controladores, modelos y servicios. En este contexto, las rutas definen los endpoints y los permisos por rol, los controladores validan la entrada con la librería zod y los modelos ejecutan SQL directo con la librería pg, sin ORM. Asimismo, un middleware verifica el token de sesión, con vigencia de 8 horas, y otro restringe cada operación según el rol. Por su parte, las contraseñas se almacenaron con hash mediante bcryptjs, y la librería helmet añadió cabeceras de seguridad HTTP. De esta manera, todos los errores llegan a un manejador central que responde con un mensaje en español y, si corresponde, con el detalle por campo.

La API se agrupó en cinco módulos con 19 endpoints, resumidos en la Tabla 3. Además, el archivo Backend/API.md especificó el método, los parámetros, el cuerpo, la respuesta de ejemplo, los códigos de error y el rol requerido de cada endpoint. Por otra parte, los scripts db:init y db:seed crean el esquema y cargan un administrador y tres soluciones de ejemplo, con documentación completa, parcial y pendiente. Por consiguiente, el indicador de cobertura muestra datos variados desde la primera ejecución.

Tabla 3

Módulos de la API REST

Módulo	**Endpoints**	**Rol requerido**
Autenticación	POST /auth/register, POST /auth/login, GET /auth/me	Público; GET /auth/me requiere sesión.
Usuarios	GET /users, PATCH /users/:id/role, DELETE /users/:id	admin

Soluciones	GET /solutions, GET /solutions/coverage, GET /solutions/recent-updates, GET, POST, PUT y DELETE /solutions/:code	Lectura: con sesión. Escritura: author y admin.
Secciones de documentación	GET, PUT y DELETE /solutions/:code/documents/:type	Lectura: con sesión. Escritura: author y admin.
Componentes	POST /solutions/:code/components, DELETE /components/:id, GET /components/:id/download	Descarga: con sesión. Escritura: author y admin.

3.2 Frontend

El frontend se desarrolló en React 19 con Vite y react-router-dom, con hojas de estilo propias y sin framework de interfaz. En este contexto, la aplicación definió nueve rutas: ingreso, registro, página principal, ficha de la solución, sección de documentación, alta y edición de soluciones, administración de usuarios y página no encontrada.

Asimismo, un contexto de autenticación conserva la sesión, valida el token al abrir la aplicación y protege cada ruta según el rol. Por consiguiente, un usuario sin sesión se redirige al ingreso, y un usuario con rol insuficiente recibe la vista «Sin permiso». Además, cada vista con datos maneja cuatro estados de interfaz: carga, vacío, error con opción de reintentar y éxito.

Las decisiones de interacción aplicaron las heurísticas de Nielsen (2024), la ley de Fitts (Fitts, 1954), la ley de Hick (Hick, 1952) y el principio de proximidad de la Gestalt (Johnson, 2020), según se resume en la Tabla 4.

Tabla 4

Principios de HCI aplicados en la interfaz

Principio	**Aplicación en el prototipo**
Visibilidad del estado del sistema	Skeletons de carga, botones con «Guardando…», avisos tras cada acción e indicador de cambios sin guardar.
Control y libertad del usuario	Botón Cancelar en los modales, cierre con Escape, opción Limpiar búsqueda y filtros en la URL.
Prevención de errores	Confirmación antes de eliminar, validación en el cliente con las reglas de la API y rechazo previo de archivos de más de 50 MB.
Recuperación de errores	Mensajes junto al campo afectado, foco en el primer campo con error y botón Reintentar.
Reconocimiento antes que recuerdo	Ayudas bajo cada campo y vista previa del Markdown.
Ley de Fitts	Objetivos de 44 px como mínimo y tarjetas que se pulsan completas.
Ley de Hick	Tres tarjetas de tipo y un buscador en la página principal; dos acciones por modal.
Proximidad (Gestalt)	Una tarjeta por bloque de información; ayuda, contador y error junto a cada campo.

4 Implementación y evaluación

4.1 Implementación

El prototipo se implementó en un entorno local en Windows, con la base de datos y el almacenamiento de objetos en la nube de Neon. En este contexto, el backend atendió en el puerto 3000 y el frontend en el puerto 5173, con un proxy de Vite que redirige las peticiones de /api hacia el backend. Asimismo, el README del repositorio documentó la instalación, las variables de entorno, la carga de datos de ejemplo y la obtención de cada rol. Por otra parte, el prototipo se demostró en funcionamiento durante la presentación en clase; sin embargo, no se publicó en un servicio de alojamiento web por limitaciones de tiempo.

4.2 Evaluación del prototipo

La evaluación se realizó mediante pruebas funcionales controladas sobre el backend en local, con datos de prueba que se eliminaron al finalizar. De acuerdo con la Tabla 5, se ejecutaron 67 casos de validaciones, autenticación, autorización e integración de la API, y los 67 resultaron aprobados. Asimismo, la colección de Postman del repositorio cubrió los mismos flujos con verificaciones del código y del contenido de cada respuesta.

Tabla 5

Resumen de las pruebas ejecutadas

Categoría	**Ejecutados**	**Aprobados**	**Fallidos**
Ingreso de datos y validaciones	25	25	0
Autenticación y autorización	17	17	0
Integración de la API	25	25	0

Total	67	67	0

5 Conclusiones

El prototipo de alta fidelidad cumplió los requerimientos funcionales del caso de estudio, con operaciones CRUD sobre soluciones, documentación, componentes y usuarios, y con control de acceso por rol. En este contexto, la evolución desde el wireframe y el mockup mantuvo la estructura validada en las fases anteriores e incorporó cambios justificados por principios de HCI y de accesibilidad. Asimismo, las 67 pruebas ejecutadas confirmaron el comportamiento esperado de la API, y la única falla detectada se corrigió antes de la entrega.

Sin embargo, la validación completa de la experiencia de usuario requiere la ejecución de las pruebas de accesibilidad y de diseño responsivo, así como pruebas con usuarios reales del portal. Por consiguiente, la siguiente iteración debe incorporar esas pruebas, la publicación del prototipo en un servicio de alojamiento y el inicio de sesión único previsto para el producto final. De esta manera, el portal quedará en condiciones de una implementación piloto con los consultores de la organización.

6 Referencias

Fitts, P. M. (1954). The information capacity of the human motor system in controlling the amplitude of movement. *Journal of Experimental Psychology, 47*(6), 381–391. https://doi.org/10.1037/h0055392

Hick, W. E. (1952). On the rate of gain of information. *Quarterly Journal of Experimental Psychology, 4*(1), 11–26. https://doi.org/10.1080/17470215208416600

Johnson, J. (2020). *Designing with the mind in mind: Simple guide to understanding user interface design guidelines* (3.ª ed.). Morgan Kaufmann.

Kirkpatrick, A., O'Connor, J., Campbell, A., & Cooper, M. (Eds.). (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. World Wide Web Consortium. https://www.w3.org/TR/WCAG21/

Nielsen, J. (2024, 30 de enero). *10 usability heuristics for user interface design*. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

Rogers, Y., Sharp, H., & Preece, J. (2023). *Interaction design: Beyond human-computer interaction* (6.ª ed.). Wiley.